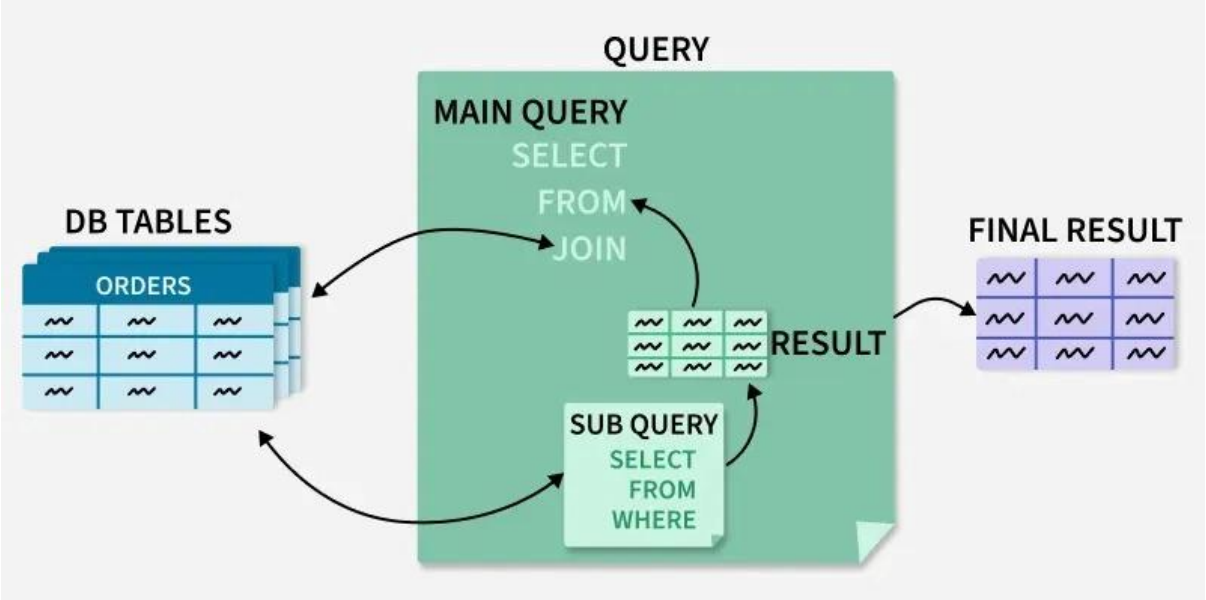
ADVANCED SQL QUERY SYSTEM

WEEK-10

1. Subqueries and Nested Queries

A subquery is a query written inside another SQL query.

The inner query is executed first, and its result is used by the outer query.



Types of Subqueries

Type	Description
Single-row subquery	Returns one value
Multiple-row subquery	Returns multiple values
Nested subquery	A subquery inside another subquery
Correlated subquery	Inner query depends on the outer query

Example

Find orders whose amount is greater than the average order amount.

```
SQL> SELECT *
  2  FROM Orders
  3  WHERE Total_Amount > (
  4      SELECT AVG(Total_Amount)
  5      FROM Orders
  6  );

  ORDER_ID CUSTOMER_ID ORDER_DAT TOTAL_AMOUNT
---------- ----------- --------- ------------
       501         201 06-OCT-26        84999
```

Explanation:

SELECT AVG(Total_Amount) FROM Orders;

calculates the average order amount.

The outer query then displays orders whose Total_Amount is greater than that average.

2. Find Products Above Average Price

A subquery can be used to compare each product's price with the average product price. Assuming the Product table contains:

- Product_ID
- Product_Name
- Price

```
SQL> SELECT *
  2  FROM Product
  3  WHERE Price > (
  4      SELECT AVG(Price)
  5      FROM Product
  6  );

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
 SELLER_ID CATEGORY_ID      PRICE
---------- ---------- ----------
BRAND
--------------------------------------------------------------------------------
     STOCK
----------
       101
iPhone 15
         1          1      74999

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
 SELLER_ID CATEGORY_ID      PRICE
---------- ---------- ----------
BRAND
--------------------------------------------------------------------------------
     STOCK
----------
Apple
        25
```

2.1 Products Below Average Price

```
SQL> SELECT *
  2  FROM Product
  3  WHERE Price < (
  4      SELECT AVG(Price)
  5      FROM Product
  6  );

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
 SELLER_ID CATEGORY_ID      PRICE
---------- ---------- ----------
BRAND
--------------------------------------------------------------------------------
     STOCK
----------
       103
OnePlus Nord CE4
         3          1      24999

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
 SELLER_ID CATEGORY_ID      PRICE
---------- ---------- ----------
BRAND
--------------------------------------------------------------------------------
     STOCK
```

3. Identify Customers with Maximum Purchases

We can use SUM() along with a subquery to identify the customer who has the highest total purchase amount.

```
SQL> SELECT C.Customer_ID,
  2         C.First_Name,
  3         C.Last_Name,
  4         SUM(O.Total_Amount) AS Total_Purchase
  5  FROM Customer C
  6  JOIN Orders O
  7  ON C.Customer_ID = O.Customer_ID
  8  GROUP BY C.Customer_ID, C.First_Name, C.Last_Name
  9  HAVING SUM(O.Total_Amount) = (
 10      SELECT MAX(Total_Purchase)
 11      FROM (
 12          SELECT SUM(Total_Amount) AS Total_Purchase
 13          FROM Orders
 14          GROUP BY Customer_ID
 15      )
 16  );

CUSTOMER_ID FIRST_NAME
----------- --------------------------------------------------
LAST_NAME                                          TOTAL_PURCHASE
-------------------------------------------------- --------------
        201 Arun
Kumar                                                       84999
```

4. Complex Business Queries

Subqueries can be combined with joins and aggregate functions to solve business-related problems.

4.1 Customers Who Placed Orders Above Average Order Value

```
SQL> SELECT C.Customer_ID,
  2         C.First_Name,
  3         C.Last_Name,
  4         O.Order_ID,
  5         O.Total_Amount
  6  FROM Customer C
  7  JOIN Orders O
  8  ON C.Customer_ID = O.Customer_ID
  9  WHERE O.Total_Amount > (
 10      SELECT AVG(Total_Amount)
 11      FROM Orders
 12  )
 13  ORDER BY O.Total_Amount DESC;

CUSTOMER_ID FIRST_NAME
----------- --------------------------------------------------
LAST_NAME                                          ORDER_ID TOTAL_AMOUNT
-------------------------------------------------- ---------- ------------
        201 Arun
Kumar                                                 501        84999
```

5. Find Products Whose Sales Are Above Average

The total sales of each product can be calculated from Order_Details.

```
SQL> SELECT P.Product_ID,
  2         P.Product_Name,
  3         SUM(OD.Subtotal) AS Total_Sales
  4  FROM Product P
  5  JOIN Order_Details OD
  6  ON P.Product_ID = OD.Product_ID
  7  GROUP BY P.Product_ID, P.Product_Name
  8  HAVING SUM(OD.Subtotal) > (
  9      SELECT AVG(Product_Sales)
 10      FROM (
 11          SELECT SUM(Subtotal) AS Product_Sales
 12          FROM Order_Details
 13          GROUP BY Product_ID
 14      )
 15  )
 16  ORDER BY Total_Sales DESC;

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------
TOTAL_SALES
-----------
       101
iPhone 15
     159998
```

6. Customer Purchase Report

A detailed customer purchase report can combine multiple tables.

```
SQL> SELECT C.Customer_ID,
  2         C.First_Name,
  3         C.Last_Name,
  4         COUNT(O.Order_ID) AS Total_Orders,
  5         SUM(O.Total_Amount) AS Total_Purchase_Amount,
  6         AVG(O.Total_Amount) AS Average_Order_Amount
  7  FROM Customer C
  8  JOIN Orders O
  9  ON C.Customer_ID = O.Customer_ID
 10  GROUP BY C.Customer_ID,
 11           C.First_Name,
 12           C.Last_Name
 13  ORDER BY Total_Purchase_Amount DESC;

CUSTOMER_ID FIRST_NAME
----------- --------------------------------------------------
LAST_NAME                                          TOTAL_ORDERS
-------------------------------------------------- ------------
TOTAL_PURCHASE_AMOUNT AVERAGE_ORDER_AMOUNT
--------------------- --------------------

        201 Arun
Kumar                                                         1
                84999                84999

        202 Priya
Sharma                                                        1
                64999                64999

CUSTOMER_ID FIRST_NAME
----------- --------------------------------------------------
LAST_NAME                                          TOTAL_ORDERS
-------------------------------------------------- ------------
TOTAL_PURCHASE_AMOUNT AVERAGE_ORDER_AMOUNT
```

Report Contains

- Customer ID
- Customer Name
- Number of orders
- Total purchase amount
- Average order value

7. Advanced Product Sales Report

This report combines product and order details.

```
SQL> SELECT P.Product_ID,
  2         P.Product_Name,
  3         P.Brand,
  4         P.Price,
  5         SUM(OD.Quantity) AS Total_Quantity_Sold,
  6         SUM(OD.Subtotal) AS Total_Sales_Amount,
  7         AVG(OD.Unit_Price) AS Average_Selling_Price,
  8         COUNT(OD.Order_ID) AS Total_Orders
  9  FROM Product P
 10  JOIN Order_Details OD
 11  ON P.Product_ID = OD.Product_ID
 12  GROUP BY P.Product_ID,
 13           P.Product_Name,
 14           P.Brand,
 15           P.Price
 16  ORDER BY Total_Sales_Amount DESC;

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
BRAND
--------------------------------------------------------------------------------
     PRICE TOTAL_QUANTITY_SOLD TOTAL_SALES_AMOUNT AVERAGE_SELLING_PRICE
---------- ------------------- ------------------ ---------------------
TOTAL_ORDERS
------------
       101
iPhone 15
Apple
```

Purpose

This report helps the business identify:

- Number of orders for each product
- Total quantity sold
- Total sales
- Average selling price
- Products generating higher sales

8. Category-wise Advanced Sales Report

Category-wise sales can be generated by combining Category, Product, and Order_Details.

```
SQL> SELECT C.Category_ID,
  2         C.Category_Name,
  3         COUNT(DISTINCT P.Product_ID) AS Total_Products,
  4         SUM(OD.Quantity) AS Total_Quantity_Sold,
  5         SUM(OD.Subtotal) AS Total_Sales_Amount,
  6         AVG(OD.Unit_Price) AS Average_Selling_Price
  7  FROM Category C
  8  JOIN Product P
  9  ON C.Category_ID = P.Category_ID
 10  JOIN Order_Details OD
 11  ON P.Product_ID = OD.Product_ID
 12  GROUP BY C.Category_ID, C.Category_Name
 13  ORDER BY Total_Sales_Amount DESC;

CATEGORY_ID
-----------
CATEGORY_NAME
--------------------------------------------------------------------------------
TOTAL_PRODUCTS TOTAL_QUANTITY_SOLD TOTAL_SALES_AMOUNT AVERAGE_SELLING_PRICE
-------------- ------------------- ------------------ --------------------
          1
Mobiles
             2                   3             224997                74999
```